Caderno de Exercícios de C#

Propostas de resolução — do iniciante ao avançado

Este caderno acompanha o **Guia C# do Iniciante ao Avançado**. Os exercícios estão organizados por dificuldade e seguem a progressão dos temas do guia. A proposta é tentar resolver cada problema antes de consultar documentação ou procurar soluções. Os enunciados não incluem respostas completas, para permitir a prática autónoma.

Como utilizar este caderno

1. Crie uma aplicação de consola com `dotnet new console -n NomeDoExercicio` ou organize vários exercícios num único projecto.
2. Leia o enunciado e identifique os dados de entrada, o processamento e os resultados esperados.
3. Resolva primeiro a versão mais simples; depois melhore a validação e a organização do código.
4. Execute o programa com exemplos normais, valores-limite e entradas inválidas.
5. Nos exercícios intermédios e avançados, escreva testes automatizados.
6. Registe dúvidas e decisões, e volte a refactorizar o código quando aprender novos conceitos.

Convenções

- **Obrigatório:** resultado mínimo esperado.
- **Desafio:** extensão opcional para aprofundar o tema.
- **Critérios de conclusão:** lista de aspectos que deve verificar antes de considerar o exercício terminado.

Nível 1 — Primeiros passos

Exercício 1 — Mensagem de boas-vindas

Crie um programa que pergunte o nome do utilizador e apresente uma mensagem personalizada.

Obrigatório: ler o nome com `Console.ReadLine` e apresentar a resposta com interpolação de strings.

Desafio: tratar o caso em que o utilizador não escreve nada e pedir novamente o nome.

Critérios de conclusão: a mensagem inclui o nome introduzido; uma entrada vazia não causa falha.

Exercício 2 — Calculadora de idade

Pergunte o ano de nascimento e calcule a idade aproximada no ano actual.

Obrigatório: converter a entrada para inteiro e apresentar a idade.

Desafio: rejeitar anos futuros e permitir que o programa seja executado repetidamente.

Critérios de conclusão: entradas não numéricas são tratadas; o programa não aceita um ano impossível.

Exercício 3 — Conversor de temperaturas

Converta uma temperatura em graus Celsius para Fahrenheit e Kelvin.

Use as fórmulas:

- Fahrenheit = Celsius × 9 / 5 + 32
- Kelvin = Celsius + 273,15

Obrigatório: apresentar os dois resultados com duas casas decimais.

Desafio: permitir escolher a conversão inversa a partir de Fahrenheit.

Critérios de conclusão: as fórmulas estão correctas e os resultados são apresentados de forma legível.

Exercício 4 — Área e perímetro

Peça ao utilizador o comprimento e a largura de um rectângulo. Calcule a área e o perímetro.

Obrigatório: apresentar os dois valores.

Desafio: recusar valores nulos ou negativos e permitir decimais.

Exercício 5 — Par ou ímpar

Leia um número inteiro e determine se é par ou ímpar.

Obrigatório: utilizar o operador resto `%` e uma condição.

Desafio: aceitar vários números até o utilizador escrever `sair`.

Exercício 6 — Maior de três números

Leia três números e apresente o maior.

Obrigatório: resolver com `if` / `else`.

Desafio: resolver também usando `Math.Max` e comparar a legibilidade das duas versões.

Exercício 7 — Classificação de nota

Leia uma nota de 0 a 20 e apresente uma classificação textual definida por si, por exemplo: insuficiente, suficiente, bom ou muito bom.

Obrigatório: validar se a nota está entre 0 e 20.

Desafio: criar uma tabela de limites configurável e permitir classificar várias notas.

Exercício 8 — Contagem crescente e decrescente

Peça um número inteiro positivo e mostre a contagem de 1 até esse número e, depois, de volta até 1.

Obrigatório: utilizar ciclos `for`.

Desafio: repetir o exercício com `while` e comparar as soluções.

Nível 2 — Condições, ciclos e métodos

Exercício 9 — Tabuada

Peça um número e mostre a respectiva tabuada de 1 a 10.

Obrigatório: usar um ciclo e apresentar cada multiplicação.

Desafio: permitir escolher o limite final da tabuada e gerar uma pequena tabela para vários números.

Exercício 10 — Jogo de adivinhação

O programa escolhe um número aleatório entre 1 e 100. O utilizador tenta adivinhá-lo, recebendo a indicação “mais alto” ou “mais baixo”.

Obrigatório: terminar quando acertar e contar as tentativas.

Desafio: limitar as tentativas, permitir começar uma nova partida e validar entradas inválidas.

Critérios de conclusão: o número aleatório está dentro do intervalo; a contagem de tentativas é correcta.

Exercício 11 — Número primo

Peça um inteiro e determine se é primo.

Obrigatório: tratar correctamente números menores do que 2.

Desafio: optimizar a verificação para testar divisores apenas até à raiz quadrada do número.

Exercício 12 — Sequência de Fibonacci

Peça quantos termos pretende gerar e apresente os primeiros termos da sequência de Fibonacci.

Obrigatório: validar que a quantidade é positiva e usar um ciclo.

Desafio: separar o cálculo num método que devolve uma lista de `long`; investigar o limite do tipo numérico.

Exercício 13 — Calculadora com métodos

Crie métodos separados para somar, subtrair, multiplicar e dividir.

Obrigatório: apresentar um menu e chamar o método correspondente.

Desafio: validar divisão por zero e permitir operações sucessivas até à opção de saída.

Exercício 14 — Conversor de unidades

Crie um menu que converta quilómetros em milhas, quilogramas em libras e Celsius em Fahrenheit.

Obrigatório: dividir a aplicação em métodos e apresentar resultados com precisão adequada.

Desafio: acrescentar conversões inversas e permitir regressar ao menu principal.

Exercício 15 — Validador de palavra-passe

Peça uma palavra-passe e verifique se cumpre regras definidas: comprimento mínimo, pelo menos uma maiúscula, uma minúscula e um algarismo.

Obrigatório: apresentar quais os critérios que falharam.

Desafio: incluir um carácter especial e criar um método que devolva uma lista de problemas encontrados.

Nível 3 — Arrays e colecções

Exercício 16 — Estatísticas de notas

Leia uma quantidade de notas e guarde-as numa lista. Apresente a média, a nota mais alta e a mais baixa.

Obrigatório: validar as notas no intervalo de 0 a 20.

Desafio: contar quantas notas estão acima da média e ordenar a lista.

Exercício 17 — Remover duplicados

Dada uma lista de números introduzida pelo utilizador, produza outra lista sem duplicados.

Obrigatório: manter a ordem da primeira ocorrência.

Desafio: resolver com `HashSet<int>` e explicar se a ordem é preservada pela estrutura escolhida.

Exercício 18 — Contador de palavras

Leia uma frase e conte quantas vezes aparece cada palavra, ignorando maiúsculas/minúsculas e pontuação básica.

Obrigatório: apresentar cada palavra e respectiva contagem.

Desafio: ordenar o resultado por frequência decrescente e depois alfabeticamente.

Exercício 19 — Lista de compras

Implemente um menu para adicionar artigos, remover artigos, listar artigos e sair.

Obrigatório: utilizar `List<string>` e validar opções do menu.

Desafio: impedir artigos vazios, procurar sem distinguir maiúsculas de minúsculas e permitir limpar a lista.

Exercício 20 — Agenda de contactos

Guarde contactos com nome, telefone e endereço de correio electrónico.

Obrigatório: usar um `Dictionary` ou uma lista de objectos e permitir adicionar e pesquisar contactos.

Desafio: permitir editar e apagar um contacto, tratando nomes repetidos.

Exercício 21 — Fila de atendimento

Simule uma fila de atendimento. Permita adicionar pessoas à fila, atender a próxima e consultar quem está à espera.

Obrigatório: usar `Queue<T>`.

Desafio: informar quando a fila está vazia e atribuir um número de atendimento.

Exercício 22 — Histórico de navegação

Simule um histórico de páginas visitadas e a operação de voltar à página anterior.

Obrigatório: usar `Stack<T>`.

Desafio: manter uma pilha separada para a operação de avançar.

Nível 4 — Classes e programação orientada a objectos

Exercício 23 — Classe `Pessoa`

Crie uma classe `Pessoa` com nome e idade. Valide os dados no construtor e disponibilize um método que indique se a pessoa é maior de idade.

Obrigatório: encapsular os dados e impedir idades negativas.

Desafio: adicionar uma data de nascimento e calcular a idade com base na data actual.

Exercício 24 — Conta bancária

Crie uma classe `ContaBancaria` com titular, saldo e métodos para depositar e levantar dinheiro.

Obrigatório: impedir depósitos negativos e levantamentos superiores ao saldo.

Desafio: registar movimentos com data, descrição e montante; escrever testes para cada

regra.

Exercício 25 — Produto e carrinho

Crie classes `Produto` e `Carrinho`. Um produto tem nome e preço; o carrinho contém vários produtos e calcula o total.

Obrigatório: impedir preços negativos e permitir adicionar/remover produtos.

Desafio: aplicar descontos através de uma abstracção como `IDesconto`.

Exercício 26 — Herança de formas

Crie uma classe abstracta `Forma` com um método para calcular a área. Implemente `Rectangulo`, `Circulo` e `Triangulo`.

Obrigatório: guardar formas numa colecção do tipo base e calcular todas as áreas através de polimorfismo.

Desafio: implementar também o perímetro e decidir como representar formas com dimensões inválidas.

Exercício 27 — Sistema de notificações

Defina uma interface `INotificador` com um método para enviar uma mensagem. Implemente notificações na consola e por correio electrónico simulado (sem enviar mensagens reais).

Obrigatório: escrever uma classe que utilize a interface sem depender de uma implementação concreta.

Desafio: criar um terceiro notificador e substituir implementações em testes.

Exercício 28 — Biblioteca

Modele livros, leitores e empréstimos. Um leitor não pode requisitar um livro já emprestado.

Obrigatório: definir classes e regras de domínio claras.

Desafio: acrescentar datas de empréstimo/devolução, limite de livros por leitor e testes das regras.

Nível 5 — Excepções, ficheiros e JSON

Exercício 29 — Leitor de ficheiros seguro

Peça o caminho de um ficheiro de texto e apresente o respectivo conteúdo.

Obrigatório: tratar ficheiro inexistente e erros de leitura com mensagens claras.

Desafio: apresentar o número de linhas, palavras e caracteres.

Exercício 30 — Diário pessoal

Crie uma aplicação que permita escrever entradas datadas e guardá-las num ficheiro.

Obrigatório: acrescentar entradas sem apagar as anteriores.

Desafio: pesquisar entradas por data ou palavra.

Exercício 31 — Persistência de tarefas em JSON

Crie uma classe `Tarefa` com identificador, descrição, estado concluído e data de criação. Guarde e carregue uma lista de tarefas em JSON.

Obrigatório: tratar o caso de o ficheiro ainda não existir.

Desafio: lidar com JSON inválido, criar cópias de segurança e separar a persistência da lógica do menu.

Exercício 32 — Importador de CSV

Leia um ficheiro CSV simples com nome, categoria e valor. Calcule totais por categoria.

Obrigatório: validar linhas mal formadas e apresentar um resumo.

Desafio: lidar correctamente com campos entre aspas e delimitadores dentro de texto; considere uma biblioteca CSV adequada para produção.

Nível 6 — LINQ, genéricos e funções

Exercício 33 — Pesquisa de produtos com LINQ

Dada uma lista de produtos, filtre os que custam menos de um valor introduzido, ordene-os por preço e seleccione apenas o nome e o preço.

Obrigatório: usar `Where`, `OrderBy` e `Select`.

Desafio: agrupar por categoria e calcular o preço médio de cada grupo.

Exercício 34 — Análise de vendas

Crie uma lista de vendas com data, produto, quantidade e preço unitário.

Obrigatório: calcular facturação total, produto mais vendido e vendas por mês.

Desafio: comparar os resultados de uma consulta LINQ com ciclos explícitos e considerar o tamanho dos dados.

Exercício 35 — Repositório genérico em memória

Crie uma classe genérica `Repositorio<T>` que permita adicionar, obter, listar e remover elementos.

Obrigatório: armazenar elementos numa colecção interna.

Desafio: definir uma interface comum para entidades com identificador e impedir identificadores duplicados.

Exercício 36 — Funções como parâmetros

Crie um método que receba uma lista de números e um `Func<int, bool>` para filtrar os elementos.

Obrigatório: utilizá-lo para obter números pares e números positivos.

Desafio: criar versões genéricas com `Func<T, bool>`.

Exercício 37 — Notificações por eventos

Crie uma classe que publique um evento quando uma tarefa é concluída. Outra classe deve subscrever o evento e apresentar uma notificação.

Obrigatório: demonstrar a publicação e a subscrição.

Desafio: remover a subscrição e testar o comportamento com vários subscritores.

Nível 7 — Assincronia e APIs

Exercício 38 — Leitura assíncrona

Leia um ficheiro de texto com `ReadAllTextAsync` e apresente o conteúdo.

Obrigatório: usar `async` e `await` e tratar erros de I/O.

Desafio: aceitar um `CancellationToken` e permitir cancelar a operação.

Exercício 39 — Cliente HTTP

Utilize `HttpClient` para obter dados de uma API pública de demonstração.

Obrigatório: fazer um pedido assíncrono e apresentar parte da resposta.

Desafio: definir timeout, tratar respostas não bem-sucedidas, desserializar JSON para um tipo próprio e permitir cancelamento. Não coloque segredos no código.

Exercício 40 — Pedidos em paralelo

Obtenha dados de vários endereços Web independentes e aguarde-os com `Task.WhenAll`.

Obrigatório: associar cada resposta ao endereço correcto.

Desafio: limitar o número de operações simultâneas e tratar falhas individuais sem perder todos os resultados.

Exercício 41 — Fila de tarefas assíncronas

Crie um processador que receba várias tarefas de trabalho e as execute de forma assíncrona.

Obrigatório: registar o início, o fim e eventuais erros de cada trabalho.

Desafio: adicionar cancelamento cooperativo e um limite de concorrência.

Nível 8 — Qualidade, testes e arquitectura

Exercício 42 — Testar a calculadora

Escreva testes unitários para uma calculadora com soma, subtracção, multiplicação e divisão.

Obrigatório: testar valores positivos, negativos, zero e divisão por zero.

Desafio: testar limites numéricos e decidir se a divisão por zero devolve um resultado especial ou lança uma excepção.

Exercício 43 — Testar a conta bancária

Escreva testes para depósitos, levantamentos, saldo insuficiente e montantes inválidos.

Obrigatório: verificar estado final e excepções esperadas.

Desafio: testar uma sequência de operações e garantir que o histórico de movimentos corresponde ao saldo.

Exercício 44 — Separar interface e lógica

Pegue num exercício de menu e mova a lógica para classes ou serviços independentes da consola.

Obrigatório: a lógica deve poder ser testada sem interagir com `Console.ReadLine`.

Desafio: introduzir interfaces para persistência e usar uma implementação em memória nos testes.

Exercício 45 — Configuração e logging

Transforme uma aplicação de consola numa aplicação pequena configurável, com parâmetros num ficheiro e registos de execução.

Obrigatório: não guardar palavras-passe ou chaves no repositório.

Desafio: distinguir ambientes de desenvolvimento e produção e evitar registar dados pessoais.

Projectos integradores

Projecto A — Gestor de tarefas

Construa uma aplicação de consola que permita criar, listar, concluir, editar e apagar tarefas.

Requisitos mínimos:

- Cada tarefa tem identificador, título, descrição, estado e data de criação.
- Os dados são guardados em JSON.
- A aplicação valida comandos e entradas.
- A lógica de tarefas está separada da apresentação na consola.
- Existem testes para as regras principais.

Extensões: pesquisa, prioridades, datas limite, filtros LINQ, exportação CSV e cancelamento de tarefas.

Projecto B — Gestão de biblioteca

Construa um sistema para gerir livros, leitores e empréstimos.

Requisitos mínimos:

- Registar livros e leitores.
- Emprestar e devolver livros.
- Impedir empréstimos inválidos.
- Guardar dados localmente.
- Testar as regras de disponibilidade e limites.

Extensões: relatórios de empréstimos, pesquisa por autor, atrasos e notificações simuladas.

Projecto C — Catálogo de despesas

Construa uma aplicação que registe despesas com data, categoria, descrição e valor.

Requisitos mínimos:

- Adicionar e consultar despesas.
- Calcular totais por período e categoria.
- Guardar e carregar os dados.
- Validar valores e datas.

Extensões: importação CSV, relatórios mensais e exportação JSON. Evite tratar este exercício como aconselhamento financeiro: é apenas uma aplicação de registo.

Projecto D — Cliente de serviço Web

Crie uma aplicação que consuma uma API pública, converta a resposta JSON em objectos e apresente resultados pesquisáveis.

Requisitos mínimos:

- Utilizar `HttpClient` de forma assíncrona.
- Tratar erros de rede e respostas inválidas.
- Permitir cancelar pedidos.
- Separar cliente HTTP, modelos e apresentação.
- Testar a lógica sem depender da rede real.

Rubrica de auto-avaliação

Classifique cada dimensão de 1 a 4:

Dimensão	1 — A melhorar	2 — Básico	3 — Consistente	4 — Sólido
Correcção	Não cobre os casos principais	Resolve o caso normal	Resolve casos normais e limites	Tem comportamento claro e verificado
Validação	Entradas não são verificadas	Valida algumas entradas	Valida entradas relevantes	Erros são tratados com clareza
Organização	Código difícil de seguir	Algumas partes separadas	Métodos e tipos bem organizados	Responsabilidades claras e baixo acoplamento
Testes	Sem testes	Testes manuais ocasionais	Testes automatizados principais	Casos normais, limites e falhas cobertos
Legibilidade	Nomes e fluxo confusos	Legível com esforço	Nomes e estrutura claros	Simples, consistente e fácil de manter

Não é necessário obter a classificação máxima em todos os exercícios. O objectivo é reconhecer o que já domina e escolher o próximo tema a praticar.

Registo de progresso

Exercício/projecto	Data	Concluído?	O que aprendi	Próximo passo

Sugestão de percurso

Comece pelos exercícios 1 a 8 e avance apenas quando conseguir explicar o código que escreveu. Depois, escolha pelo menos dois exercícios de cada nível. Termine com um projecto integrador, incluindo validação e testes. Para cada problema, tente primeiro uma solução simples; em seguida, reveja-a com os conceitos novos do guia.